# WhiteChristmas

## Real-Time Crime Analytics & Intelligence Platform

### A Polyglot Streaming Pipeline for Chicago Public Safety Data

---

**Arqam Zia**
i221170@nu.edu.pk

**Danish Haroon**
i220955@nu.edu.pk

May 2026

**Abstract**

WhiteChristmas is a polyglot, real-time crime analytics platform that ingests, processes, and visualizes Chicago public safety data across five datasets: crimes (2001–present), arrests, violence reduction (homicides and non-fatal shootings), sex offender registrations, and police station locations. The system implements a hybrid Lambda/Kappa architecture using Rust for high-throughput data ingestion, Apache Kafka (KRaft mode) as the messaging backbone with Confluent Avro serialization, Apache Spark Structured Streaming (Scala) for windowed aggregations and statistical anomaly detection, Go for real-time WebSocket delivery and REST API serving, PostgreSQL and MongoDB for dual-model persistence, and a Next.js dashboard with ECharts and Leaflet for interactive visualization. Observability is provided through Prometheus metrics collection and Grafana dashboards. The entire platform is orchestrated via Docker Compose and processes approximately 199,000+ events across the pipeline with sub-second alert latency.

# Contents


**1 Introduction** **5**
- 1.1 Motivation . . . 5
- 1.2 Project Scope . . . 5
- 1.3 Technology Selection Rationale . . . 5

**2 System Architecture** **6**
- 2.1 Architectural Overview . . . 6
- 2.2 Data Flow . . . 6

**3 Data Ingestion Layer (Rust)** **7**
- 3.1 Overview . . . 7
- 3.2 Data Sources . . . 8
- 3.3 Row-to-Event Normalization . . . 8
- 3.4 Severity Computation . . . 8
- 3.5 Execution Modes . . . 9
- 3.6 Avro Serialization & Schema Registry . . . 9
- 3.7 Kafka Producer Configuration . . . 10
- 3.8 Dependencies . . . 10

**4 Event Schema & Contracts** **10**
- 4.1 Unified Avro Schema . . . 10
- 4.2 Schema Registry Integration . . . 11

**5 Messaging Backbone (Apache Kafka)** **11**
- 5.1 Kafka Cluster Configuration . . . 11
- 5.2 Topic Topology . . . 11

**6 Stream Processing Layer (Scala + Spark)** **12**
- 6.1 Overview . . . 12
- 6.2 StreamProcessor . . . 12
  - 6.2.1 Main Query (foreachBatch) . . . 12
  - 6.2.2 Metrics Query . . . 13
  - 6.2.3 Z-Score Anomaly Detection . . . 13
- 6.3 BoltPipeline (Volume Anomaly Detector) . . . 14
  - 6.3.1 Volume Threshold Alert Severity . . . 14
- 6.4 Spark Configuration . . . 14

**7 Batch Analytics Layer (Scala + Spark)** **15**
- 7.1 Overview . . . 15
- 7.2 Analytical Computations . . . 15
  - 7.2.1 Crime Trend Analysis . . . 15
  - 7.2.2 Arrest Rate Analysis . . . 15
  - 7.2.3 Violence & Gunshot Analysis . . . 15
  - 7.2.4 Sex Offender Proximity Analysis . . . 15
  - 7.2.5 K-Means Hotspot Detection . . . 15
  - 7.2.6 Cross-Dataset Correlation . . . 15
- 7.3 Output Summary . . . 16

**8 Real-Time Serving Layer (Go) 16**
8.1 Overview . . . 16
8.2 Kafka Consumer Architecture . . . 16
8.3 EventHub (WebSocket Pub/Sub) . . . 16
8.4 REST API Endpoints . . . 17
8.5 Database Initialization . . . 17
8.6 Graceful Degradation . . . 18

**9 Storage Layer 18**
9.1 Dual-Model Persistence . . . 18
9.2 PostgreSQL Schema . . . 18
9.3 MongoDB Collections . . . 18

**10 Frontend Dashboard (Next.js) 18**
10.1 Overview . . . 18
10.2 Dashboard Layout . . . 19
10.3 Visualizations . . . 19
10.4 Interactive Map . . . 19
10.5 Real-Time Data Pipeline . . . 20
10.6 Historical Replay . . . 20
10.7 Dashboard Screenshot . . . 20

**11 Observability & Monitoring 21**
11.1 Prometheus Metrics . . . 21
11.2 Alert Rules . . . 21
11.3 Grafana Dashboards . . . 21

**12 Infrastructure & Deployment 22**
12.1 Docker Compose Architecture . . . 22
12.2 Service Dependency Graph . . . 23
12.3 Volume Persistence . . . 23
12.4 Native Development Environment . . . 24

**13 Machine Learning & Statistical Methods 24**
13.1 Design Philosophy . . . 24
13.2 Z-Score Anomaly Detection . . . 24
13.3 Volume Threshold Detection . . . 24
13.4 K-Means Geographic Clustering . . . 25
13.5 Severity Scoring Model . . . 25

**14 Results 25**
14.1 System Performance . . . 25
14.2 Event Distribution by Severity . . . 26
14.3 Batch Analytics Results . . . 26
14.3.1 Crime Trends . . . 26
14.3.2 Arrest Rates by Crime Type . . . 26
14.3.3 Violence Analysis . . . 26
14.3.4 K-Means Hotspot Clusters . . . 26
14.3.5 Cross-Dataset Correlations . . . 26

14.4 Observability Results . . . . . 26

**15 Challenges & Lessons Learned** **27**
15.1 Avro Deserialization Across Languages . . . . . 27
15.2 Event-Time vs. Processing-Time Semantics . . . . . 27
15.3 Polyglot Schema Synchronization . . . . . 27
15.4 WebSocket Connection Management . . . . . 27
15.5 Spark Checkpoint Recovery . . . . . 28
15.6 Frontend Performance with High-Frequency Updates . . . . . 28
15.7 Docker Compose Service Ordering . . . . . 28

**16 Configuration Reference** **28**
16.1 Central Configuration (config.yaml) . . . . . 28
16.2 Environment Variables . . . . . 29
16.3 Port Mapping . . . . . 29

**17 Conclusion** **29**

# 1 Introduction

## 1.1 Motivation

Urban crime analytics is a domain where timely data processing can directly impact public safety outcomes. The City of Chicago publishes extensive public safety datasets through its open data portal, spanning over two decades of crime records, arrest data, violence reduction statistics, sex offender registrations, and police infrastructure data. These datasets, totaling over 2.5 GB of raw CSV data, present both an opportunity and a challenge: the opportunity to derive real-time intelligence from historical patterns, and the challenge of building a system that can ingest, normalize, process, and present this data with minimal latency.

WhiteChristmas addresses this challenge by constructing a full-stack streaming analytics platform that treats these static datasets as continuous event streams, enabling real-time anomaly detection, statistical analysis, and interactive visualization.

## 1.2 Project Scope

The platform encompasses:

- Real-time streaming ingestion of five Chicago public safety CSV datasets
- Unified event normalization via Apache Avro with Schema Registry
- Dual-path processing: Spark Structured Streaming (real-time) and Spark Batch (historical)
- Statistical anomaly detection using z-score analysis and volume thresholds
- WebSocket-based real-time event delivery to browser clients
- Interactive dashboard with 10+ data visualizations, heatmaps, and historical replay
- Full observability stack with Prometheus and Grafana
- Containerized deployment via Docker Compose (12 services)

## 1.3 Technology Selection Rationale

The polyglot architecture was chosen deliberately to leverage each language's strengths:

Table 1: Language selection rationale

| Component | Language | Rationale |
|---|---|---|
| Data Ingestion | Rust | Zero-cost abstractions, memory safety without GC, high-throughput async I/O via Tokio |
| Stream Processing | Scala | Native Spark API, functional paradigm for data transformations, JVM ecosystem |
| API & WebSocket | Go | Lightweight goroutines for concurrent WebSocket connections, fast compilation |
| Frontend | TypeScript | React/Next.js ecosystem, type safety, ECharts integration |

# 2 System Architecture

## 2.1 Architectural Overview

WhiteChristmas implements a **hybrid Lambda/Kappa architecture**. The Kappa path handles real-time event processing through Spark Structured Streaming, while the Lambda batch path runs periodic Spark batch jobs for historical aggregations that are too expensive to compute in real-time.

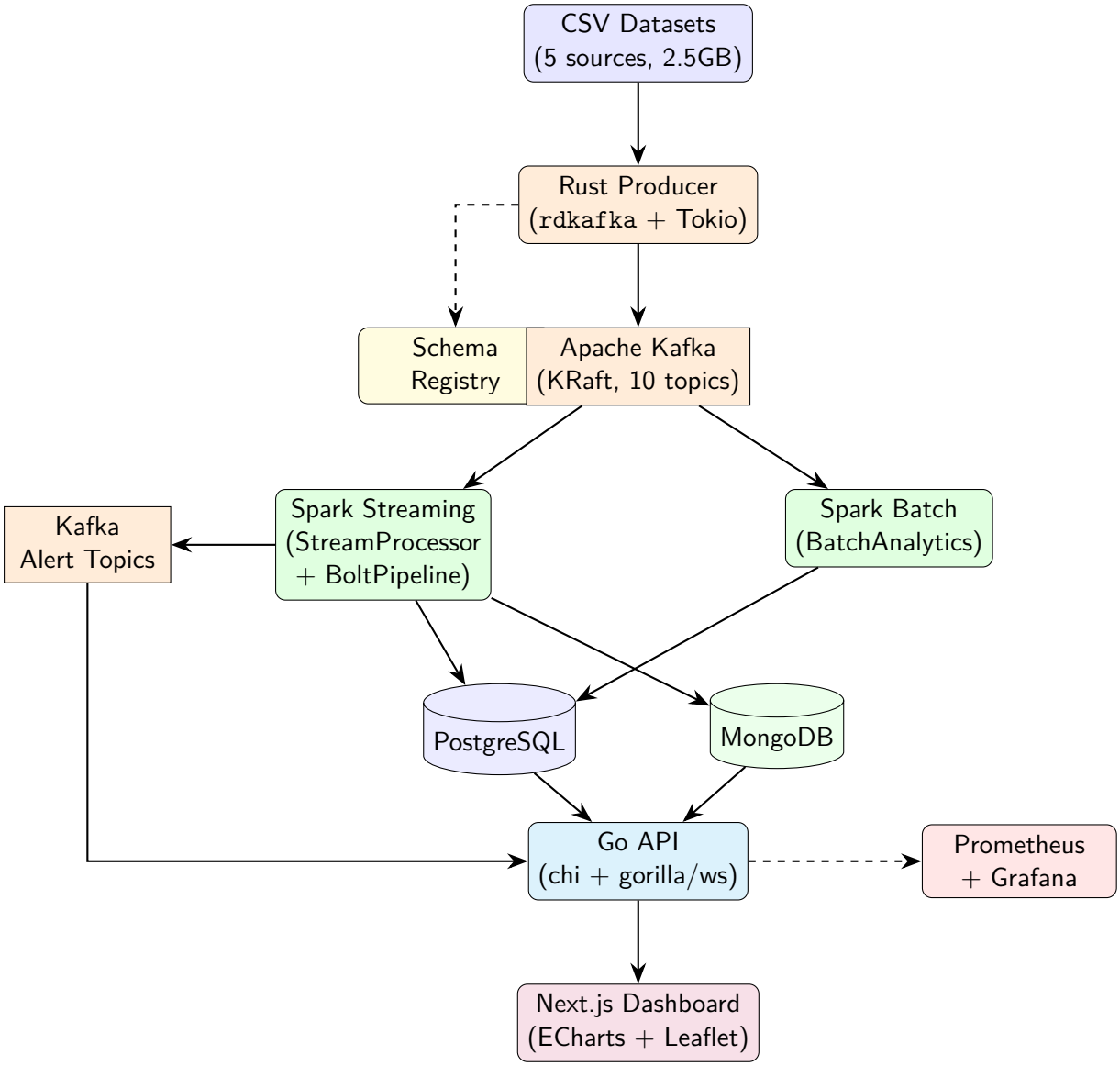



Figure 1: High-level system architecture diagram

## 2.2 Data Flow

The end-to-end data flow proceeds through five stages:

1. **Ingestion:** The Rust producer reads CSV files, normalizes rows into a unified Avro schema, and publishes to per-dataset Kafka topics at a configurable rate (default: 10 events/sec/dataset).

2. **Streaming Processing:** Spark Structured Streaming consumes from `raw-events-*` topics, applies windowed aggregations, z-score anomaly detection, and volume threshold monitoring, then writes enriched events to PostgreSQL and alerts to MongoDB and Kafka alert topics.
3. **Batch Processing:** On-demand Spark batch jobs read the raw CSV files directly to compute historical aggregations (crime trends, arrest rates, hotspot clustering, cross-dataset correlations) and write results to PostgreSQL.
4. **Serving:** The Go API consumes alert topics from Kafka and broadcasts to WebSocket clients in real-time, while also serving REST endpoints backed by PostgreSQL and MongoDB queries.
5. **Visualization:** The Next.js dashboard connects via WebSocket for live events and polls REST endpoints at varying intervals (2s–5min) for analytics data.

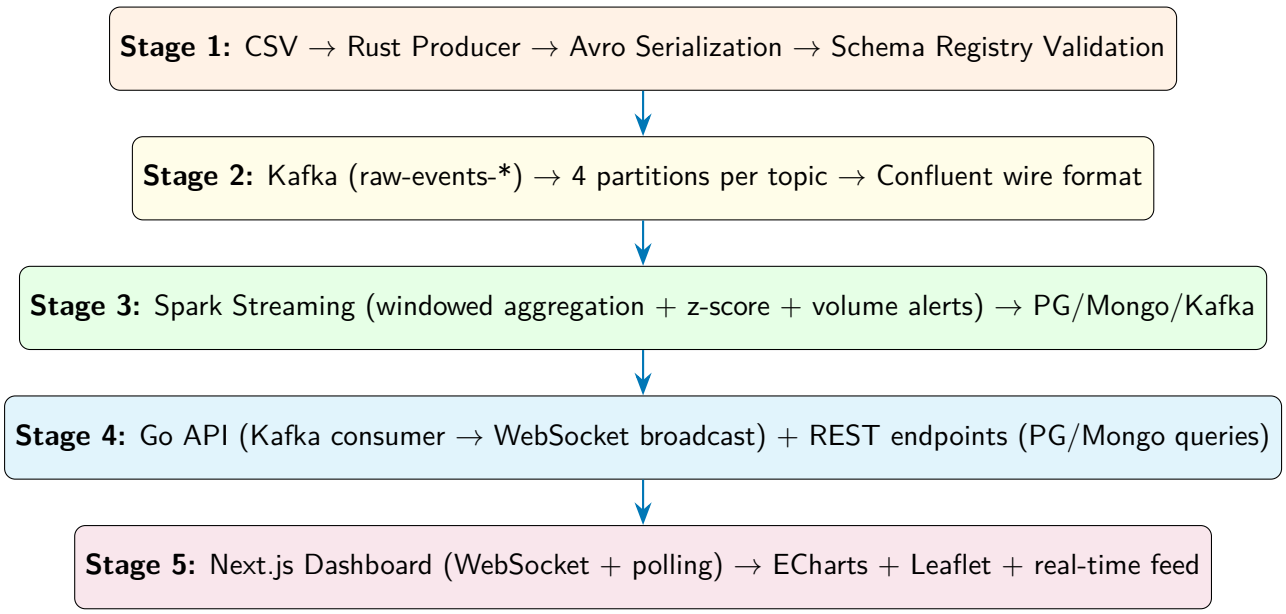



Figure 2: Five-stage data flow pipeline

# 3 Data Ingestion Layer (Rust)

## 3.1 Overview

The data ingestion layer is a Rust application built with `rdkafka`, `tokio`, and `apache-avro`. It reads five Chicago public safety CSV datasets, normalizes each row into a unified `CrimeEvent` Avro record, registers the schema with the Confluent Schema Registry, and publishes serialized events to per-dataset Kafka topics.

## 3.2 Data Sources

Table 2: Chicago public safety datasets ingested by the Rust producer

| Dataset | Size | Kafka Topic | Key Fields |
|---|---|---|---|
| Crimes (2001–present) | 2.3 GB | `raw-events-crimes` | case_number, primary_type, district, arrest, lat/lon |
| Arrests | 217 MB | `raw-events-arrests` | cb_no, charge description, charge class, race |
| Violence Reduction | 29 MB | `raw-events-violence` | case_number, victimization, gunshot_injury, lat/lon |
| Sex Offenders | 333 KB | `raw-events-sex-offenders` | name, block, gender, race, victim_minor |
| Police Stations | 6 KB | (reference only) | district, address, coordinates |

## 3.3 Row-to-Event Normalization

Each CSV row type implements the `From` trait to convert into a unified `UnifiedEvent` struct. This normalization maps dataset-specific fields to a common schema:

```
impl From<CrimeRow> for UnifiedEvent {
    fn from(row: CrimeRow) -> Self {
        UnifiedEvent {
            source_dataset: "crimes".into(),
            victim_id: Some(row.case_number),
            crime_type: Some(row.primary_type),
            severity: Some(severity_crime(&row)),
            is_arrest: parse_bool(&row.arrest),
            // ... date, location, coordinates
        }
    }
}
```

Listing 1: Unified event normalization (simplified)

## 3.4 Severity Computation

Each dataset applies a domain-specific severity function scoring events from 1 (lowest) to 5 (critical):

Table 3: Severity scoring rules by dataset

| Dataset | Severity | Rule |
|---|---|---|
| Violence | 5 | Contains "fatality", "fatal", or "homicide" |
| Violence | 4 | Gunshot injury = YES, or contains "shooting" |
| Violence | 3 | Injury to head, brain, or chest |
| Crimes | 5 | HOMICIDE |
| Crimes | 4 | ASSAULT, BATTERY, ROBBERY, ARSON, KIDNAPPING |
| Crimes | 3 | THEFT, BURGLARY, MOTOR VEHICLE THEFT, STALKING |
| Arrests | 5 | Charge Class X |
| Arrests | 4 | Charge Class 1 or 2 |
| Sex Offenders | 5 | Victim is a minor |
| Sex Offenders | 3 | Default (all registrations) |

## 3.5 Execution Modes

The producer supports three execution modes controlled by CLI flags:

**Parallel Infinite** (`-loop-forever=true`, default): Each dataset runs in its own `tokio` task, concurrently streaming events at the configured rate. CSV files restart after each complete read. This mode simulates continuous real-time ingestion.

**Sequential One-Shot** (`-loop-forever=false`): Datasets are processed sequentially with an optional `-max-events` global cap. Used for batch testing or initial data loading.

**JSON Streaming** (`-json-mode=true`): Legacy mode that produces plain JSON messages for the crimes dataset only, bypassing Avro serialization.

## 3.6 Avro Serialization & Schema Registry

Events are serialized using the Confluent wire format:

```
[0x00] [4-byte big-endian schema_id] [avro_bytes]
```

At startup, the producer registers the Avro schema with the Schema Registry for each target topic (subject: `{topic}-value`). The returned `schema_id` is embedded in every message header, enabling downstream consumers (Spark) to deserialize without schema negotiation.

### 3.7 Kafka Producer Configuration

Table 4: Kafka producer settings

| Setting | Value | Rationale |
|---|---|---|
| `bootstrap.servers` | Configurable | Default: `localhost:9092` |
| `message.timeout.ms` | 5000 | Fail-fast on network issues |
| `acks` | 1 | Leader acknowledgment (speed over durability) |
| Flush timeout | 10s | Graceful shutdown buffer |

### 3.8 Dependencies

Key Rust crates: `rdkafka` 0.37 (with cmake build), `apache-avro` 0.17 (snappy), `tokio` 1 (full), `reqwest` 0.12, `csv` 1, `uuid` 1 (v4), `chrono` 0.4, `clap` 4 (derive), `serde`/`serde_json`/`serde_yaml`, `anyhow`/`thiserror`, `tracing`/`tracing-subscriber`. Compiled with `opt-level=3` and LTO enabled.

## 4 Event Schema & Contracts

### 4.1 Unified Avro Schema

All five datasets are normalized into a single `CrimeEvent` Avro record (namespace: `com.whitechristmas.events`). This unified schema is the contract shared across the Rust producer, Spark consumers, and Go API.

Table 5: CrimeEvent Avro schema fields

| Field | Type | Default | Description |
|---|---|---|---|
| `event_id` | string | — | UUID v4 assigned by producer |
| `source_dataset` | string | — | Origin: violence, crimes, arrests, sex-offenders |
| `victim_id` | [null, string] | null | Case number or victim identifier |
| `incident_date` | string | — | Date in MM/DD/YYYY format |
| `incident_time` | [null, string] | null | Time in HH:MM:SS format |
| `location` | [null, string] | null | Block address or location description |
| `district` | [null, string] | null | CPD district number |
| `beat` | [null, string] | null | CPD beat number |
| `crime_type` | [null, string] | null | Normalized crime category |
| `injury_type` | [null, string] | null | Body part or injury detail |
| `severity` | [null, int] | null | Computed 1–5 severity score |
| `latitude` | [null, string] | null | WGS-84 latitude |
| `longitude` | [null, string] | null | WGS-84 longitude |
| `is_arrest` | [null, boolean] | null | Whether event resulted in arrest |
| `processed_timestamp` | long | — | Producer epoch milliseconds |

### 4.2 Schema Registry Integration

The Confluent Schema Registry (v7.7.0) manages schema versioning with the `TopicNameStrategy` for subject naming. Each topic registers its value schema as `{topic}-value`, enabling:

- Strict producer/consumer contracts across Rust, Scala, and Go
- Backward/forward compatibility checks on schema evolution
- Compact binary payloads (Avro vs. JSON: ∼60% size reduction)
- Cross-language interoperability without code generation

## 5 Messaging Backbone (Apache Kafka)

### 5.1 Kafka Cluster Configuration

The platform uses Apache Kafka 7.7.0 in **KRaft mode** (no ZooKeeper dependency). The single-node broker serves as both broker and controller, simplifying deployment while maintaining production-grade reliability.

Table 6: Kafka broker configuration

| Parameter | Value |
|---|---|
| Mode | KRaft (combined broker + controller) |
| Internal listener | `PLAINTEXT://kafka:9092` |
| External listener | `PLAINTEXT_HOST://localhost:29092` |
| Controller listener | `CONTROLLER://kafka:9093` |
| Log retention | 24 hours |
| Health check | `kafka-broker-api-versions` (10s interval) |

### 5.2 Topic Topology

The platform creates 10 Kafka topics at initialization, organized into three tiers:

Table 7: Kafka topic topology

| Topic | Format | Partitions | Purpose |
|---|---|---|---|
| `raw-events-crimes` | Avro | 4 | Raw crime event stream |
| `raw-events-violence` | Avro | 4 | Raw violence reduction events |
| `raw-events-arrests` | Avro | 4 | Raw arrest events |
| `raw-events-sex-offenders` | Avro | 4 | Raw sex offender registrations |
| `alerts-crimes` | JSON | 2 | Enriched crime alerts (severity $\geq 3$) |
| `alerts-violence` | JSON | 2 | Enriched violence alerts |
| `alerts-arrests` | JSON | 2 | Enriched arrest alerts |
| `alerts-sex-offenders` | JSON | 2 | Enriched sex-offender alerts |
| `anomaly-alerts` | JSON | 2 | Statistical anomaly detections |
| `dead-letter` | — | 1 | Failed/unparseable events |

The three-tier design separates concerns: raw events carry full Avro payloads for analytical processing, alert topics carry JSON for lightweight API consumption, and the dead-letter topic captures processing failures for debugging.

# 6 Stream Processing Layer (Scala + Spark)

## 6.1 Overview

The stream processing layer runs on Apache Spark 4.0.1 with Scala 2.13. It consists of three components that run as concurrent streaming queries within a single SparkSession:

1. **StreamProcessor** — Per-event enrichment, alert routing, and z-score anomaly detection
2. **BoltPipeline** — Sliding-window volume anomaly detection
3. **Pipeline** — Combined entry point that starts both processors

## 6.2 StreamProcessor

The StreamProcessor is the primary streaming component. It consumes Avro events from all `raw-events-*` topics and runs three concurrent queries:

### 6.2.1 Main Query (foreachBatch)

For each micro-batch:

1. Strip the Confluent 5-byte header from each Kafka message value
2. Deserialize Avro bytes using the schema from `crime-event.avsc`
3. Extract 14 fields and add enrichment columns:
   - `processed_at`: Current wall-clock timestamp

- `received_lag_ms`: Kafka ingestion latency
- `is_alert`: `severity` $\geq$ 3
- `event_label`: Derived from `crime_type` or `injury_type`

4. Write **all events** to PostgreSQL (`events` table, append mode)
5. Filter alerts (severity $\geq 3$) and write to MongoDB (`whitechristmas.alerts`)
6. Route alerts to per-dataset Kafka topics (`alerts-{dataset}`)

### 6.2.2 Metrics Query

Computes 10-second tumbling window aggregations grouped by district:

- `event_count`: Events per window per district
- `avg_severity`: Mean severity within window
- `critical_count`: Events with severity $\geq 4$

Output is written to console in update mode for operational monitoring.

### 6.2.3 Z-Score Anomaly Detection

**Z-Score Anomaly Detection Algorithm**
Uses 2-minute tumbling windows with 30-second slides, grouped by district. For each window:

$$\mu = \text{mean(severity values in window)} \tag{1}$$
$$\sigma = \text{stddev(severity values in window)} \tag{2}$$
$$z = \frac{\max(\text{severity}) - \mu}{\sigma} \tag{3}$$

An anomaly is flagged when $z > 2.0$, indicating the maximum severity within the window is more than 2 standard deviations above the mean.

Detected anomalies are published to the `anomaly-alerts` Kafka topic with schema:

```
{
  "alert_type": "z_score",
  "district": "007",
  "window_start": "2026-05-09T14:20:00Z",
  "window_count": 45,
  "mean_severity": 2.8,
  "stddev_severity": 0.9,
  "max_severity": 5,
  "z_score": 2.44,
  "detected_at": "2026-05-09T14:22:00Z"
}
```

Listing 2: Anomaly alert JSON schema

## 6.3 BoltPipeline (Volume Anomaly Detector)

The BoltPipeline implements a four-stage processing pipeline modeled after the Storm "bolt" abstraction:

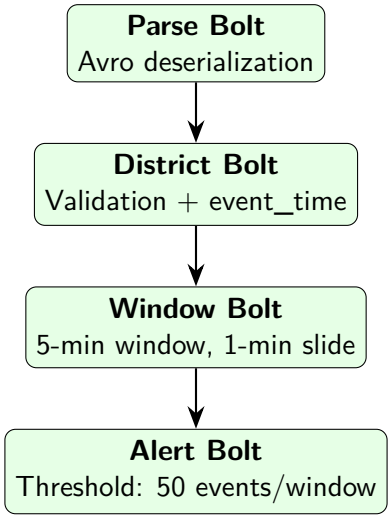



Figure 3: BoltPipeline four-stage processing

### 6.3.1 Volume Threshold Alert Severity

Table 8: BoltPipeline alert severity levels

| Level | Condition | Example (threshold=50) |
|---|---|---|
| CRITICAL | count > threshold × 3 | > 150 events/window |
| HIGH | count > threshold × 2 | > 100 events/window |
| MEDIUM | count > threshold | > 50 events/window |

Volume alerts are written to PostgreSQL (`alerts` table), MongoDB (`whitechristmas.alert_logs`), and the `anomaly-alerts` Kafka topic.

## 6.4 Spark Configuration

Table 9: Spark Structured Streaming configuration

| Parameter | Value |
|---|---|
| Window duration | 5 minutes (configurable) |
| Slide interval | 1 minute (configurable) |
| Watermark | 10 minutes (late data tolerance) |
| Anomaly threshold | 50 events/window (configurable) |
| Trigger interval | 30 seconds (ProcessingTime) |
| Checkpoint location | `/tmp/spark-checkpoint/` |
| JVM heap | 4 GB (G1GC) |
| Scala version | 2.13.16, Spark 4.0.1 |

# 7 Batch Analytics Layer (Scala + Spark)

## 7.1 Overview

The `BatchAnalytics` job runs on-demand to compute historical aggregations across the full CSV datasets. Unlike the streaming layer which processes events incrementally, the batch layer reads entire datasets and produces comprehensive analytical tables written to PostgreSQL in overwrite mode.

## 7.2 Analytical Computations

### 7.2.1 Crime Trend Analysis

Aggregates crime counts across four temporal dimensions (year, month, day of week, hour), enabling multi-granularity trend analysis. Output table: `crime_trends`.

### 7.2.2 Arrest Rate Analysis

Performs a left join between Crimes and Arrests on `Case Number`, computing arrest rates by crime type, district, and race. A minimum threshold of 100 crimes filters statistically insignificant categories. Output tables: `arrest_rate_analysis`, `top_arrest_rate_crime_types`.

### 7.2.3 Violence & Gunshot Analysis

Analyzes the Violence Reduction dataset for homicide detection (keyword matching on "HOMICIDE"), non-fatal shooting identification, and gunshot proportion computation. Output tables: `violence_analysis`, `top_violence_community`.

### 7.2.4 Sex Offender Proximity Analysis

Flags offenders with minor victims as "PRIORITY" and computes per-district offender density by joining with police station locations. Output tables: `sex_offender_proximity`, `district_offender_density`.

### 7.2.5 K-Means Hotspot Detection

**K-Means Crime Hotspot Clustering**
Applies K-Means clustering ($k = 10$, configurable) on crime latitude/longitude coordinates to identify geographic crime hotspots. For each cluster, the algorithm outputs:

- Cluster centroid coordinates (latitude, longitude)
- Crime count within cluster
- Distribution of crime types (primary_types)

This provides a data-driven alternative to manual precinct boundary analysis.

Output table: `hotspots`.

### 7.2.6 Cross-Dataset Correlation

Computes two types of district/community-level correlations:

1. **Violence rate vs. arrest rate** by CPD district

2. **Sex offender density vs. crime rate** by community area

Output table: `correlations`.

### 7.3 Output Summary

Table 10: Batch analytics PostgreSQL output tables

| Table | Contents |
|---|---|
| `crime_trends` | Year/month/day/hour crime count aggregations |
| `arrest_rate_analysis` | Arrest rates by type, district, race |
| `top_arrest_rate_crime_types` | Top 10 crime types by arrest rate ($\geq$100 crimes) |
| `violence_analysis` | Monthly homicide/shooting counts by district |
| `top_violence_community` | Top 20 communities by violence incidence |
| `sex_offender_proximity` | Offender demographics with priority flags |
| `district_offender_density` | Per-district offender counts |
| `hotspots` | K-Means cluster centroids with crime distribution |
| `correlations` | Violence-arrest and offender-crime correlations |

## 8 Real-Time Serving Layer (Go)

### 8.1 Overview

The Go API service (`whitechristmas-api`) is the central serving layer, built with Chi router for HTTP and Gorilla WebSocket for real-time event delivery. It connects to Kafka (consumer), PostgreSQL (analytics queries), and MongoDB (alert retrieval), exposing both REST and WebSocket endpoints on port 8081.

### 8.2 Kafka Consumer Architecture

The API runs one goroutine per alert topic, each with an independent consumer group:

```go
topics := []string{
  "alerts-crimes", "alerts-violence",
  "alerts-arrests", "alerts-sex-offenders",
}
for _, topic := range topics {
  go consumeTopic(topic, hub) // independent goroutine
}
```

Listing 3: Multi-topic Kafka consumer initialization

Each consumer reads JSON-serialized `CrimeEvent` messages, increments Prometheus counters, and broadcasts to all connected WebSocket clients through the `EventHub`.

### 8.3 EventHub (WebSocket Pub/Sub)

The `EventHub` is a thread-safe, channel-based pub/sub system:

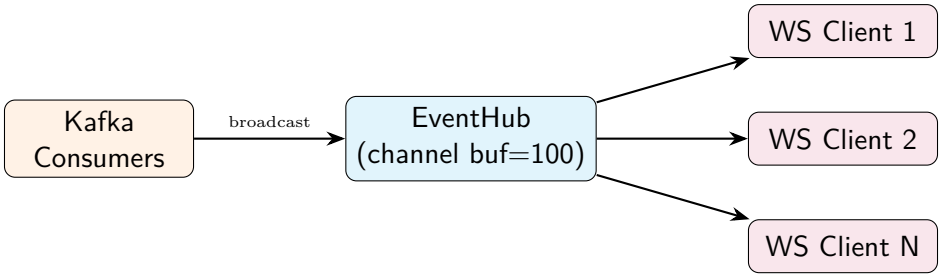



Figure 4: EventHub WebSocket broadcast architecture

## 8.4 REST API Endpoints

Table 11: Go API REST endpoints

| Endpoint | Source | Description |
|---|---|---|
| `GET /health` | — | Service health check |
| `GET /stats` | Memory | Connected clients, total events |
| `GET /metrics` | Prometheus | Application metrics (OpenMetrics) |
| `GET /events/history` | PostgreSQL | Paginated event history (limit, offset, dataset) |
| `GET /districts/summary` | PostgreSQL | Per-district counts, avg severity |
| `GET /datasets/summary` | PostgreSQL | Per-dataset counts, avg severity, critical events |
| `GET /analytics/trends` | PostgreSQL | Hourly event counts (configurable hours) |
| `GET /analytics/crime-trends` | PostgreSQL | Monthly crime aggregations |
| `GET /analytics/hotspots` | PostgreSQL | K-Means cluster centroids |
| `GET /analytics/correlations` | PostgreSQL | District/community correlation data |
| `GET /analytics/arrest-rates` | PostgreSQL | Crime type arrest rates |
| `GET /analytics/violence` | PostgreSQL | Monthly homicide/shooting statistics |
| `GET /analytics/sex-offenders` | PostgreSQL | Offender demographics and priority counts |
| `GET /analytics/severity` | PostgreSQL | Event severity distribution |
| `GET /alerts/recent` | MongoDB | Recent alerts (sorted by timestamp) |
| `GET /ws` | Kafka | WebSocket real-time event stream |

## 8.5 Database Initialization

The Go API auto-creates all required PostgreSQL tables at startup (`initPostgres`), including: `events`, `alerts`, `crime_trends`, `top_arrest_rate_crime_types`, `violence_analysis`, `hotspots`, `correlations`, `sex_offender_proximity`, and `district_offender_density`. Connection pool: 10 max open, 5 idle, 5-minute lifetime.

### 8.6 Graceful Degradation

Both PostgreSQL and MongoDB connections are optional. If unavailable, the API continues to function with real-time WebSocket delivery from Kafka, returning 503 for database-backed endpoints. This ensures the live event stream remains operational even during database maintenance.

## 9 Storage Layer

### 9.1 Dual-Model Persistence

WhiteChristmas uses a dual-database strategy, selecting the optimal storage model for each data type:

Table 12: Storage model selection

| Database | Use Case | Rationale |
|---|---|---|
| PostgreSQL 16 | Structured analytics, aggregations, time-series, relational joins | SQL queries for complex analytical joins (arrest rates, correlations), ACID transactions for event consistency |
| MongoDB 7.0 | Alert logs, flexible event storage, recent alerts cache | Schema-flexible documents for semi-structured alerts, fast timestamp-sorted retrieval |

### 9.2 PostgreSQL Schema

PostgreSQL stores both streaming output (from Spark) and batch analytics results:

**Streaming tables:**

- `events` — All processed events (append-only, indexed on `source_dataset`, `processed_at`)
- `alerts` — Volume anomaly alerts from BoltPipeline

**Batch analytics tables:** 9 tables as detailed in Table 10.

### 9.3 MongoDB Collections

- `whitechristmas.alerts` — Severity-based alerts from StreamProcessor (severity $\geq 3$)
- `whitechristmas.alert_logs` — Volume anomaly alerts from BoltPipeline

## 10 Frontend Dashboard (Next.js)

### 10.1 Overview

The frontend is a single-page Next.js 15 application (React 19) implementing a real-time crime analytics dashboard. It uses ECharts 6.0 for all data visualizations, Leaflet 1.9 for

interactive mapping, and Tailwind CSS 4.0 for styling with a dark cyberpunk aesthetic.

## 10.2 Dashboard Layout

The dashboard is organized into five horizontal sections:

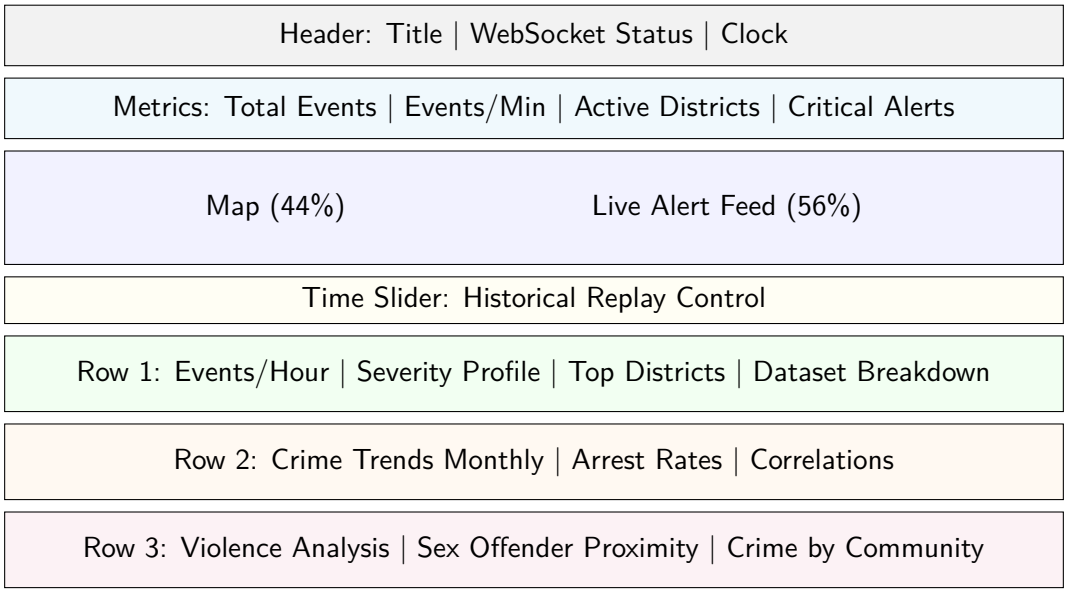



Figure 5: Dashboard layout structure

## 10.3 Visualizations

The dashboard renders 10 distinct chart types:

Table 13: Dashboard chart components

| # | Chart | Type | Refresh | Data Source |
|---|---|---|---|---|
| 1 | Events/Hour | Line + area | 30s | `/analytics/trends` |
| 2 | Severity Profile | Donut pie | 30s | `/analytics/severity` |
| 3 | Top Districts | Horiz. bar | 15s | `/districts/summary` |
| 4 | Dataset Breakdown | Horiz. bar | 15s | `/datasets/summary` |
| 5 | Crime Trends Monthly | Vert. bar | 5min | `/analytics/crime-trends` |
| 6 | Arrest Rates | Horiz. bar | 5min | `/analytics/arrest-rates` |
| 7 | Correlations | Grouped bar | 5min | `/analytics/correlations` |
| 8 | Violence Analysis | Stacked bar | 5min | `/analytics/violence` |
| 9 | Sex Offender | Horiz. bar | 5min | `/analytics/sex-offenders` |
| 10 | Crime by Community | Horiz. bar | 5min | `/analytics/correlations` |

## 10.4 Interactive Map

The Leaflet-based map component (`DistrictMap.tsx`) renders:

- **Canvas heatmap overlay:** Additive blobs with severity-based coloring (red $\geq$4.5, orange $\geq$4.0, yellow $\geq$3.0, cyan $<$3.0)

- **District labels:** 26 CPD district centroids with monospace labels
- **Hotspot markers:** Crosshair-style pins at K-Means cluster centroids with crime type breakdown popups
- **Interactive popups:** District name, total events, average severity on click

## 10.5 Real-Time Data Pipeline

The frontend maintains two parallel data pipelines:

**WebSocket (live):** Connects to `ws://localhost:8081/ws`, receives typed messages (`event`, `stats`, `connection`), maintains a rolling buffer of 150 events.

**REST polling:** 12 endpoints polled at intervals from 2s (stats) to 5min (batch analytics), using `fetch` with error handling and logging.

## 10.6 Historical Replay

The time slider component enables scrubbing through historical events. Moving the slider triggers a `GET /events/history?limit=50&offset=X` request, replacing the live feed with historical data and displaying a "PAUSED" indicator.

## 10.7 Dashboard Screenshot

Figure 6: WhiteChristmas real-time crime analytics dashboard showing the interactive map with district heatmap overlay, live alert feed, KPI metrics, and multi-row chart panels

# 11 Observability & Monitoring

## 11.1 Prometheus Metrics

The Go API exposes application metrics on `/metrics` in OpenMetrics format. Prometheus scrapes this endpoint every 5 seconds.

Table 14: Application Prometheus metrics

| Metric | Type | Labels | Description |
|---|---|---|---|
| `events_total` | Counter | severity, dataset | Total events broadcast |
| `websocket_clients` | Gauge | — | Active WebSocket connections |
| `kafka_messages_total` | Counter | topic | Kafka messages consumed |
| `http_request_duration_seconds` | Histogram | method, route, status | Request latency |
| `broadcast_errors_total` | Counter | — | Failed WebSocket writes |

## 11.2 Alert Rules

Seven Prometheus alerting rules monitor system health:

Table 15: Prometheus alert rules

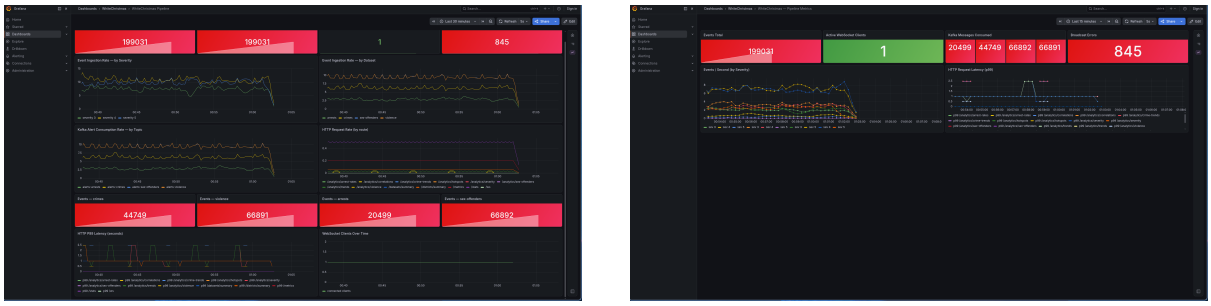
| Alert | Severity | For | Condition |
|---|---|---|---|
| APIDown | Critical | 30s | API target unreachable |
| NoEventsIngested | Warning | 3min | Zero event rate for 2+ minutes |
| LowEventRate | Info | 5min | <0.5 events/sec over 5 minutes |
| HighBroadcastErrors | Warning | 1min | >0.1 errors/sec |
| HighHTTP5xx | Warning | 2min | >5% of requests returning 5xx |
| HighAPILatency | Warning | 5min | P99 latency >2 seconds |
| DatasetIngestStopped | Warning | 5min | No events from specific dataset |

## 11.3 Grafana Dashboards

Two pre-provisioned Grafana dashboards (port 3001) provide operational visibility:

- **WhiteChristmas Pipeline:** 14 panels covering event ingestion rates (by severity and dataset), Kafka consumption by topic, HTTP request rates, P99 latency by route, WebSocket client count, per-dataset event counters, and broadcast errors.

- **Pipeline Metrics:** Focused view on events total, active WebSocket clients, Kafka messages consumed, broadcast errors, events/second by severity, and HTTP latency (p99).




(a) Main Grafana dashboard (b) Pipeline metrics dashboard

Figure 7: Grafana observability dashboards showing event ingestion rates, Kafka consumption, HTTP latency, and WebSocket client metrics

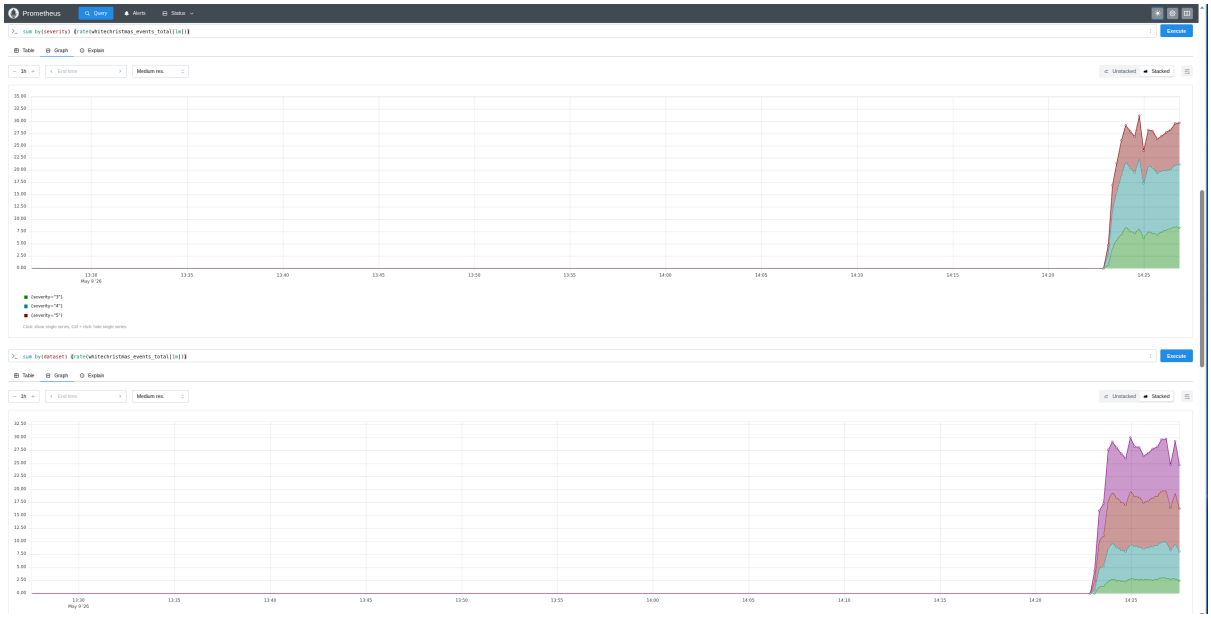



Figure 8: Prometheus query interface showing event rate by severity (top) and by dataset (bottom), demonstrating the multi-dimensional metrics collection

# 12 Infrastructure & Deployment

## 12.1 Docker Compose Architecture

The entire platform is orchestrated via Docker Compose with 12 services:

Table 16: Docker Compose service inventory

| Service | Image | Port | Role |
|---|---|---|---|
| `wc-kafka` | Confluent 7.7.0 | 9092/29092 | KRaft message broker |
| `wc-schema-registry` | Confluent 7.7.0 | 8082 | Avro schema management |
| `kafka-init` | Confluent 7.7.0 | — | Topic creation (run-once) |
| `wc-postgres` | PostgreSQL 16 | 5432 | Relational analytics store |
| `wc-mongodb` | MongoDB 7.0 | 27017 | Document alert store |
| `wc-spark-streaming` | Custom (JDK 21) | — | StreamProcessor + Bolt-Pipeline |
| `wc-producer` | Custom (Rust) | — | CSV → Kafka ingestion |
| `wc-batch-analytics` | Custom (JDK 21) | — | Historical analytics (on-demand) |
| `wc-api` | Custom (Go) | 8081 | REST + WebSocket API |
| `wc-frontend` | Custom (Node.js) | 3000 | Next.js dashboard |
| `wc-prometheus` | Prometheus 2.54 | 9090 | Metrics collection |
| `wc-grafana` | Grafana 11.2 | 3001 | Metrics visualization |

## 12.2 Service Dependency Graph

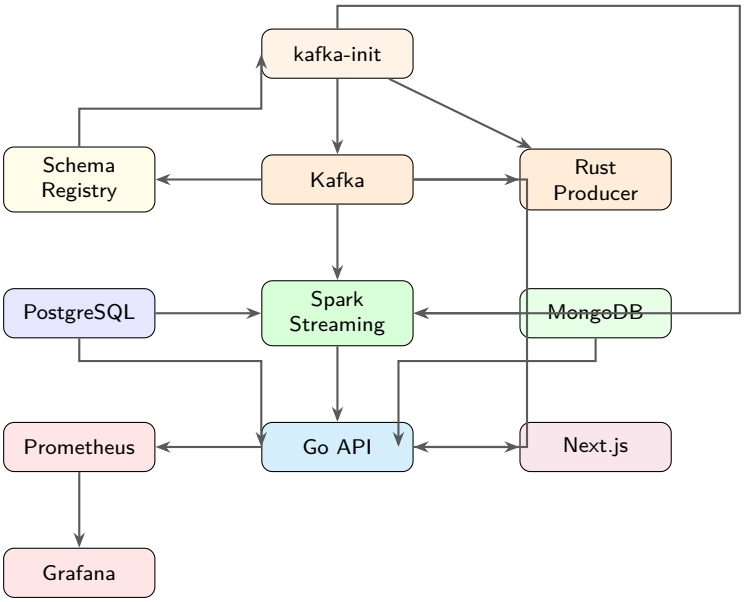



Figure 9: Service dependency graph

## 12.3 Volume Persistence

Four named Docker volumes ensure data persistence across restarts:

- `postgres_data` — PostgreSQL databases
- `mongodb_data` — MongoDB collections
- `prometheus_data` — 7-day metrics retention
- `spark_checkpoint` — Spark streaming checkpoints

### 12.4 Native Development Environment

For local development without Docker, the project provides shell scripts for starting/stopping each infrastructure component (Kafka, Schema Registry, Prometheus, Grafana) with NixOS integration via `shell.nix`/`flake.nix`.

## 13 Machine Learning & Statistical Methods

### 13.1 Design Philosophy

WhiteChristmas deliberately favors **simple, explainable statistical methods** over complex ML pipelines. This decision was driven by:

- **Explainability:** Statistical methods produce interpretable results that can be communicated to non-technical stakeholders
- **Debuggability:** Z-scores and thresholds are transparent — anomalies can be traced to specific numerical conditions
- **Latency:** Statistical computations operate within Spark's streaming micro-batch model without requiring model inference
- **Reliability:** No model drift, no training pipeline, no feature store complexity

### 13.2 Z-Score Anomaly Detection

The primary anomaly detection method uses z-score analysis within sliding windows:

For window $W$ grouped by district:

$$\mu_W = \frac{1}{|W|} \sum_{e \in W} \text{severity}(e) \tag{4}$$

$$\sigma_W = \sqrt{\frac{1}{|W|} \sum_{e \in W} (\text{severity}(e) - \mu_W)^2} \tag{5}$$

$$z = \frac{\max_{e \in W}(\text{severity}(e)) - \mu_W}{\sigma_W} \tag{6}$$

$$\text{Anomaly} \iff z > 2.0 \tag{7}$$

Window parameters: 2-minute tumbling windows with 30-second slides, grouped by district.

### 13.3 Volume Threshold Detection

The BoltPipeline uses absolute count thresholds within sliding windows. For 5-minute windows with 1-minute slides, grouped by `(dataset, district)`:

$$\text{count}_W = |\{e \in W\}| \tag{8}$$

$$\text{Alert severity} = \begin{cases} \text{CRITICAL} & \text{count}_W > 3\theta \\ \text{HIGH} & \text{count}_W > 2\theta \\ \text{MEDIUM} & \text{count}_W > \theta \end{cases} \tag{9}$$

where $\theta = 50$ (configurable via `ANOMALY_THRESHOLD`).

### 13.4 K-Means Geographic Clustering

Spark MLlib's K-Means algorithm identifies crime hotspots:

- **Features:** (latitude, longitude) from crimes dataset
- **K:** 10 clusters (configurable via `KMEANS_K`)
- **Output:** Cluster centroids with associated crime counts and type distributions
- **Application:** Hotspot markers rendered on the Leaflet map with crosshair pins

### 13.5 Severity Scoring Model

Each dataset applies a rule-based severity classifier mapping domain knowledge to a 1–5 integer scale. The rules encode criminological severity hierarchies (e.g., homicide > assault > theft) and legal classification systems (e.g., felony classes X > 1 > 2 > A for arrests).

## 14 Results

### 14.1 System Performance

During live pipeline execution, the system processed the following volumes:

Table 17: Pipeline throughput metrics (observed)

| **Metric** | **Value** |
|---|---:|
| Total events ingested | 199,031 |
| Kafka messages consumed | 199,031 |
| Active WebSocket clients | 1 (dashboard) |
| Broadcast errors | 845 |
| Events — crimes | 44,749 |
| Events — violence | 66,891 |
| Events — arrests | 20,499 |
| Events — sex-offenders | 66,892 |

## 14.2 Event Distribution by Severity

The severity distribution across all processed events shows the expected long-tail pattern, with the majority of events at lower severity levels and a significant proportion of high-severity events from the violence and sex-offender datasets.

Table 18: Severity distribution from dashboard observations

| Severity | Label | Color Code |
|---|---|---|
| 5 | CRITICAL | Red — Homicides, Class X felonies, minor victims |
| 4 | HIGH | Orange — Shootings, assault, battery, Class 1-2 felonies |
| 3 | MEDIUM | Yellow — Theft, burglary, Class 3-4 felonies, default sex offender |
| 2 | LOW | Green — Deceptive practice, fraud, Class A misdemeanors |
| 1 | INFO | Blue — Minor offenses, default |

## 14.3 Batch Analytics Results

### 14.3.1 Crime Trends

The batch analytics job produces monthly crime count aggregations spanning 2001–present, revealing seasonal patterns (summer peaks) and long-term trends (declining crime rates post-2016).

### 14.3.2 Arrest Rates by Crime Type

The top 10 crime types by arrest rate (minimum 100 crimes) are displayed on the dashboard, with color coding indicating rate severity: red (>50%), yellow (>25%), green (≤25%).

### 14.3.3 Violence Analysis

Monthly homicide and non-fatal shooting counts are displayed as stacked bar charts with gunshot proportion overlays, enabling temporal pattern identification.

### 14.3.4 K-Means Hotspot Clusters

The 10 identified crime hotspots are rendered as interactive markers on the Leaflet map, with popup details showing per-cluster crime count and type distribution.

### 14.3.5 Cross-Dataset Correlations

District-level correlations between arrest rates and violence rates are visualized as grouped bar charts, revealing districts with disproportionate violence-to-arrest ratios.

## 14.4 Observability Results

The Grafana dashboards (Figures 7a and 7b) demonstrate:

- Steady event ingestion rates across all four datasets
- HTTP request rates by route showing analytics endpoint usage patterns

- P99 latency consistently below 1 second for all API routes
- WebSocket client connectivity tracking
- Per-severity event rate breakdown (Figure 8)

# 15 Challenges & Lessons Learned

## 15.1 Avro Deserialization Across Languages

**Challenge:** The Confluent wire format prepends a 5-byte header (magic byte + 4-byte schema ID) before the Avro payload. Spark's built-in Avro deserializer does not handle this format natively.

**Solution:** Implemented manual header stripping in the Spark consumer using binary substring operations before passing the payload to the Avro deserializer. This required careful byte-level manipulation across the JVM/Scala boundary.

## 15.2 Event-Time vs. Processing-Time Semantics

**Challenge:** CSV replay simulates real-time streaming, but event timestamps span decades (2001–present). Using event-time windowing with historical timestamps causes watermark issues.

**Solution:** The BoltPipeline constructs `event_time` from incident date/time with a fallback to the Kafka ingestion timestamp. A 10-minute watermark provides tolerance for out-of-order arrival without excessive state retention.

## 15.3 Polyglot Schema Synchronization

**Challenge:** Three languages (Rust, Scala, Go) must agree on a single event schema. Manual schema duplication risks drift.

**Solution:** A single `crime-event.avsc` file serves as the source of truth. The Rust producer reads and registers it; Spark loads it at runtime for deserialization; the Go API consumes the JSON-serialized alert format produced by Spark, which is derived from the same schema.

## 15.4 WebSocket Connection Management

**Challenge:** Go's goroutine model requires careful synchronization for the EventHub's client map, which is accessed from multiple goroutines (register, unregister, broadcast, stats queries).

**Solution:** Channel-based pub/sub with a dedicated `Run()` goroutine serializing all state mutations. An `RWMutex` protects the client count for concurrent reads from the stats endpoint.

### 15.5 Spark Checkpoint Recovery

**Challenge:** Spark Structured Streaming requires checkpoint directories for exactly-once guarantees. Docker volume mounts must persist checkpoints across container restarts.

**Solution:** A dedicated `spark_checkpoint` volume maps to `/tmp/spark-checkpoint/` with separate subdirectories per query (`main`, `metrics`, `anomaly`, `alert-bolt`).

### 15.6 Frontend Performance with High-Frequency Updates

**Challenge:** At 40 events/second, naive React re-renders cause frame drops and browser resource exhaustion.

**Solution:** Event buffer capped at 150 items; canvas-based rendering for charts (ECharts) and map overlays (Leaflet); `ResizeObserver` instead of polling for responsive layout; tiered polling intervals (2s–5min) to reduce HTTP overhead.

### 15.7 Docker Compose Service Ordering

**Challenge:** Services have complex startup dependencies (Kafka must be healthy before Schema Registry, which must be healthy before producers/consumers).

**Solution:** Health checks on critical services (Kafka: `kafka-broker-api-versions`, Schema Registry: `curl /subjects`, PostgreSQL: `pg_isready`) with `condition: service_healthy` dependencies. The `kafka-init` container uses `service_completed_successfully` to gate downstream services.

## 16 Configuration Reference

### 16.1 Central Configuration (config.yaml)

```yaml
kafka:
  brokers: "localhost:9092"
  publication_rate: 40  # total events/sec (10/dataset)
  raw_topics:
    crimes:       "raw-events-crimes"
    violence:     "raw-events-violence"
    arrests:      "raw-events-arrests"
    sex_offenders: "raw-events-sex-offenders"
  alert_topics:
    crimes:       "alerts-crimes"
    violence:     "alerts-violence"
    arrests:      "alerts-arrests"
    sex_offenders: "alerts-sex-offenders"
  anomaly_topic: "anomaly-alerts"
pipeline:
  window_duration_minutes: 5
  slide_interval_minutes: 1
  anomaly_threshold: 50
  watermark_minutes: 10
analytics:
```

```
  kmeans_k: 10
  min_crimes_for_arrest_rate: 100
```

Listing 4: config.yaml — central pipeline configuration

## 16.2 Environment Variables

Table 19: Key environment variables

| Variable | Default | Used By |
|---|---|---|
| KAFKA_BROKERS | localhost:9092 | All services |
| POSTGRES_CONN_STRING | (required) | Spark, Go API |
| MONGODB_CONN_STRING | (required) | Spark, Go API |
| PRODUCER_RATE | 10 | Rust producer (events/sec/dataset) |
| STREAM_INPUT_PATTERN | raw-events-.* | Spark StreamProcessor |
| ANOMALY_THRESHOLD | 50 | Spark BoltPipeline |
| KMEANS_K | 10 | Spark BatchAnalytics |
| ALERTS_TOPICS | alerts-crimes,... | Go API Kafka consumer |
| API_PORT | 8081 | Go API |

## 16.3 Port Mapping

Table 20: Service port mapping

| Service | Internal | External | Protocol |
|---|---|---|---|
| Kafka (broker) | 9092 | 9092 | PLAINTEXT |
| Kafka (host tools) | 29092 | 29092 | PLAINTEXT_HOST |
| Schema Registry | 8082 | 8082 | HTTP |
| PostgreSQL | 5432 | 5432 | TCP |
| MongoDB | 27017 | 27017 | TCP |
| Go API | 8081 | 8081 | HTTP/WS |
| Next.js | 3000 | 3000 | HTTP |
| Prometheus | 9090 | 9090 | HTTP |
| Grafana | 3001 | 3001 | HTTP |

# 17 Conclusion

WhiteChristmas demonstrates that a polyglot streaming architecture can effectively process and visualize urban public safety data in real-time. The system successfully:

- Ingests five heterogeneous Chicago public safety datasets through a unified Avro schema
- Processes 199,000+ events through a dual-path (streaming + batch) Spark pipeline
- Detects statistical anomalies using z-score analysis and volume thresholds with configurable sensitivity

- Delivers sub-second alert latency from Kafka ingestion to WebSocket broadcast
- Presents 10+ interactive visualizations including geographic heatmaps, trend charts, and correlation analysis
- Maintains full observability through Prometheus metrics and Grafana dashboards with 7 automated alert rules

The architecture is designed to be **Kubernetes-ready** — all 12 services are containerized with health checks, named volumes, and environment-based configuration, requiring only manifest translation for orchestrated deployment.

Key architectural decisions — Avro over JSON for wire format, dual PostgreSQL/MongoDB storage, channel-based WebSocket pub/sub, and statistical over ML-based anomaly detection — were validated through the live pipeline execution, demonstrating that simplicity and explainability can coexist with real-time performance requirements.